

AWS ELB & Auto Scaling — Complete Notes

Load balancing + elasticity · Distribute traffic, scale automatically

AWS ELB & ASG

Goal High Availability

ELB ALB/NLB/GWLB

ASG Self Healing

A reader-friendly, all-in-one reference for Elastic Load Balancing & EC2 Auto Scaling.

Table of Contents

1. Why ELB & Auto Scaling
2. Load Balancer Types
3. ALB (Application Load Balancer)
4. NLB (Network Load Balancer)
5. Health Checks
6. Key ELB Concepts
7. Auto Scaling Groups (ASG)
8. Launch Templates
9. Scaling Policies
10. Lifecycle & Health
11. ELB + ASG: HA Architecture
12. Pricing
13. Common CLI Commands
14. Best Practices
15. Quick Mental Model
16. Common Interview Questions

1. Why ELB & Auto Scaling

These two services together give you **high availability** and **elasticity**:

- **Elastic Load Balancing (ELB)** spreads incoming traffic across multiple healthy targets in multiple AZs, so no single instance is a bottleneck or a single point of failure.
- **EC2 Auto Scaling (ASG)** automatically adds/removes instances to match demand and **replaces unhealthy ones** — self-healing.

 **Mental model:** the **ELB is the front door** that splits traffic; the **ASG is the engine room** that grows/shrinks the fleet and heals it. Together: traffic always reaches a healthy instance, and capacity follows demand.

2. Load Balancer Types

Type	Layer	Routes on	Best for
ALB (Application)	L7 (HTTP/HTTPS)	Host, path, headers, query, method	Web apps, microservices, containers
NLB (Network)	L4 (TCP/UDP/TLS)	IP + port	Ultra-low latency, millions of req/s, static IP
GWLB (Gateway)	L3	IP packets	Inline virtual appliances (firewalls, IDS/IPS)
CLB (Classic, legacy)	L4/L7	basic	Old accounts only — avoid for new work

Pick **ALB** for HTTP(S) apps, **NLB** for extreme performance / TCP / a static IP, **GWLB** to insert security appliances.

3. ALB (Application Load Balancer)

Operates at **Layer 7** — understands HTTP, so it can route on content.

- **Listener** — checks for connections on a port/protocol (e.g. HTTPS:443) and applies **rules**.
- **Rules** — route by **host** (`api.example.com`), **path** (`/images/*`), header, method, or query → to a **target group**.
- **Target group** — a set of targets (EC2, IP, **Lambda**, containers) with its own health check.
- Features: **path/host-based routing**, **HTTP/2 & WebSockets**, **redirects & fixed responses**, **authentication** (Cognito/OIDC), **sticky sessions**, **TLS termination** (ACM certs).

```
Client → ALB listener :443
├ rule: /api/* → target group (api service)
├ rule: /img/* → target group (image service)
└ default    → target group (web)
```

4. NLB (Network Load Balancer)

Operates at **Layer 4** (TCP/UDP/TLS) for **extreme performance** and **low latency**.

- Handles **millions of requests/sec**; very low latency.
- Provides a **static IP per AZ** (and supports Elastic IPs) — useful for allow-lists/firewalls.
- **Preserves the client source IP** to targets.
- Can do **TLS termination**; great for non-HTTP protocols, gaming, IoT, and when you need a fixed IP.

5. Health Checks

- The LB periodically probes each target; only **healthy** targets receive traffic.
- Tunables: protocol/port/path, **interval**, **timeout**, **healthy/unhealthy thresholds**.
- A failing target is **drained** (existing connections finish via *deregistration delay*) and removed from rotation; recovery re-adds it.

- ELB health checks can also tell the **ASG** to replace an instance (see §10).

6. 🧩 Key ELB Concepts

Concept	Meaning
Cross-zone load balancing	Evenly spreads traffic across targets in all AZs (default on for ALB; optional for NLB).
Sticky sessions	Pins a client to the same target via a cookie (ALB/CLB).
SSL/TLS termination	LB decrypts HTTPS using an ACM cert, forwarding HTTP to targets.
Connection draining (deregistration delay)	Lets in-flight requests finish before removing a target.
SNI	Multiple certs/domains on one ALB listener.
Access logs	Optionally log requests to S3.

7. 📈 Auto Scaling Groups (ASG)

An **ASG** manages a fleet of EC2 instances between capacity bounds.

Setting	Role
Minimum	Never go below this many instances.
Desired	The target count right now.
Maximum	Never exceed this many.
Launch Template	Defines <i>what</i> to launch (AMI, type, SG, user data).
Subnets/AZs	Spread instances across multiple AZs for HA.

- The ASG keeps the running count at **Desired**, **replaces failed instances**, and registers/deregisters them with the ELB automatically.

8. 📝 Launch Templates

- A **Launch Template** specifies the instance configuration the ASG uses: AMI, instance type, key pair, security groups, IAM role, user data, storage.
- **Versioned** (preferred over the older *launch configuration*) and supports **mixed instances** (combine types + On-Demand/Spot for cost).

9. 🎯 Scaling Policies

Policy	How it scales
Target Tracking	Keep a metric at a target (e.g. CPU = 50%) — simplest, recommended.
Step Scaling	Add/remove N instances based on alarm magnitude.
Simple Scaling	One adjustment per alarm (older).
Scheduled Scaling	Change capacity on a schedule (e.g. scale up 9am).
Predictive Scaling	ML forecasts load and pre-scales ahead of demand.

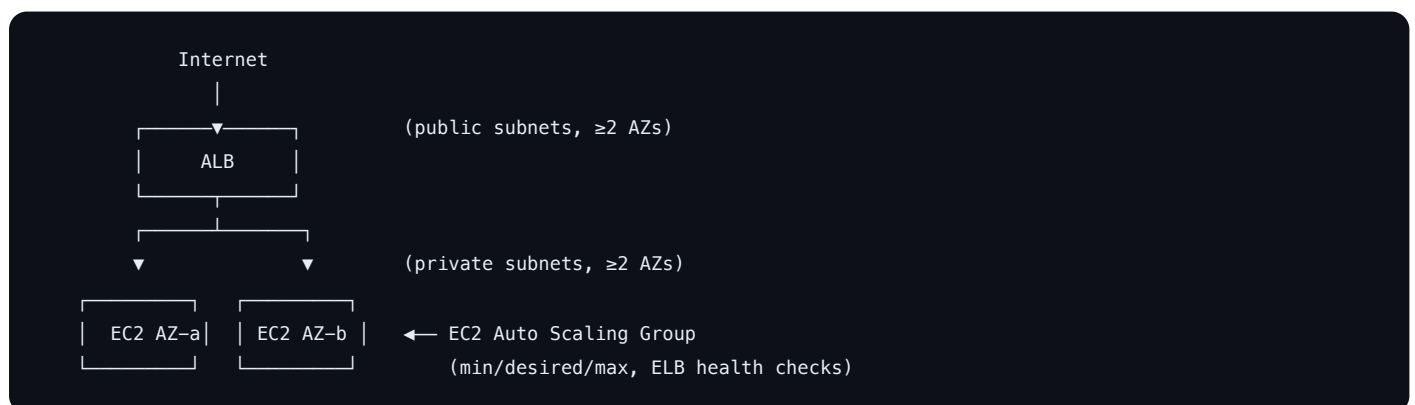
- Policies are driven by **CloudWatch alarms** (CPU, request count per target, custom metrics).
- **Cooldown** periods prevent thrashing between rapid scale events.

10. 🔄 Lifecycle & Health

- **Health check type:** **EC2** (instance status checks) or **ELB** (the load balancer's health check). Using **ELB health checks** lets the ASG replace instances the app — not just the OS — considers unhealthy.
- **Lifecycle hooks** pause instances in **Pending** / **Terminating** so you can run setup (bootstrap) or cleanup (drain, deregister) before they go live or away.
- **Termination policies** decide which instance to remove on scale-in (e.g. oldest launch template, closest to billing hour).

11. 🏛️ ELB + ASG: HA Architecture

The textbook highly-available web tier:



- ALB in **public** subnets across **≥2 AZs**; instances in **private** subnets across the same AZs.
- ASG keeps the fleet sized to load and **replaces unhealthy instances**; ELB only routes to healthy ones.
- Result: **elastic** (scales with demand) + **self-healing** (failures auto-replaced) + **multi-AZ** (survives an AZ outage).

12. 💰 Pricing

- **ELB** — per **load-balancer-hour** + **LCU/NLCU** (capacity units measuring connections, bandwidth, rules processed).
- **Auto Scaling** — **free**; you only pay for the **EC2 instances** (and EBS) it launches.
- **Data transfer** out is charged as usual.

🔧 **Cost tips:** Auto Scaling itself is free — scale in aggressively for non-prod · use **mixed instances + Spot** in the ASG for cheap capacity · right-size before scaling out · consolidate services behind one ALB (host/path routing) instead of many LBs.

13. 🖨️ Common CLI Commands

```
# Load balancer (ELBv2 = ALB/NLB)
aws elbv2 create-load-balancer --name web-alb --subnets subnet-a subnet-b
aws elbv2 create-target-group --name web-tg --protocol HTTP --port 80 --vpc-id vpc-123
aws elbv2 create-listener --load-balancer-arn <arn> --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn=<tg-arn>

# Auto Scaling
aws autoscaling create-auto-scaling-group --auto-scaling-group-name web-asg \
  --launch-template LaunchTemplateName=web-lt --min-size 2 --max-size 6 \
  --desired-capacity 2 --vpc-zone-identifier "subnet-a,subnet-b" \
  --target-group-arns <tg-arn> --health-check-type ELB
aws autoscaling put-scaling-policy --auto-scaling-group-name web-asg \
  --policy-name cpu50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{"PredefinedMetricSpecification":{"PredefinedMetricType":"ASGAverageCPUUtilization"},"'
aws autoscaling describe-auto-scaling-groups
```

14. ✅ Best Practices

- Spread the **ELB and ASG across ≥2 AZs**; put instances in **private** subnets behind the LB.
- Use **ELB health checks** on the ASG so app-level failures trigger replacement.
- Prefer **Target Tracking** scaling (simple, robust); add **Scheduled/Predictive** for known patterns.
- Use **Launch Templates + mixed instances/Spot** for cost-efficient capacity.
- Set **cooldowns** to avoid thrashing; use **lifecycle hooks** for clean bootstrap/drain.
- Terminate gracefully with **connection draining** (deregistration delay).
- Consolidate microservices behind one **ALB** via host/path rules; use **NLB** for static IP / extreme performance.

15. 🧠 Quick Mental Model

- **ELB = front door** distributing traffic to healthy targets across AZs; **ASG = engine room** scaling and healing the fleet.
- **ALB** (L7, host/path routing, web/microservices) · **NLB** (L4, ultra-fast, static IP) · **GWLb** (appliances) · **CLB** (legacy).
- ALB: **listener** → **rules** → **target groups** (each with a **health check**).

- **Health checks** keep traffic on healthy targets; **connection draining** finishes in-flight requests.
 - $ASG = \min / \text{desired} / \max + \text{launch template}$ across AZs; replaces unhealthy instances.
 - **Scaling policies:** Target Tracking (default), Step, Scheduled, Predictive — driven by CloudWatch alarms.
 - **ALB + ASG across ≥ 2 AZs = elastic + self-healing + highly available.** Auto Scaling is free; pay for instances.
-

16. ? Common Interview Questions

Rapid-fire questions interviewers ask about ELB & Auto Scaling:

- **Q: ALB vs NLB?** — ALB = Layer 7 (HTTP/HTTPS), routes on host/path/headers, for web apps/microservices. NLB = Layer 4 (TCP/UDP), ultra-low latency, millions of req/s, static IP.
 - **Q: How do you make a web app highly available?** — ALB + Auto Scaling Group across ≥ 2 AZs; instances in private subnets; ELB health checks for self-healing.
 - **Q: What does an Auto Scaling Group give you?** — Elasticity (match capacity to demand) and resilience (auto-replace unhealthy/failed instances).
 - **Q: EC2 vs ELB health check on the ASG?** — EC2 check = instance status only. ELB check = the load balancer's app-level check, so the ASG replaces instances the app considers unhealthy.
 - **Q: What scaling policies exist?** — Target Tracking (keep a metric at a target), Step, Simple, Scheduled, Predictive.
 - **Q: What is connection draining?** — Deregistration delay that lets in-flight requests finish before a target is removed.
 - **Q: What is cross-zone load balancing?** — Distributes traffic evenly across targets in all AZs (default on for ALB).
 - **Q: How does SSL termination work?** — The ALB holds an ACM cert, decrypts HTTPS, and forwards HTTP to targets (offloading TLS).
 - **Q: Does Auto Scaling cost extra?** — No — it's free; you pay only for the EC2 instances and EBS it launches.
 - **Q: How do you reduce capacity cost?** — Mixed instances with Spot in the ASG, right-sizing, and scheduled scale-in for non-prod.
-

 Notes compiled as a quick reference for AWS ELB & Auto Scaling.